



2024

高级单元测试

目录

CONTENTS

PART 01 Junit示例

PART 02 注解

PART 03 测试类与测试方法

Assert

TestCase

TestResult

TestSuite

PART 03 错误与异常处理

PART 03 批量测试

01



Junit示例



Junit示例



开展快速有效的单元测试离不开单元测试框架的支持。对于Java语言而言，JUnit就是一款十分成熟且常用的单元测试开源框架

JUnit对测试驱动的软件开发起到了非常重要的推动作用，它促进了“先测试后编码”的软件开发理念，强调首先构建测试代码，然后再构建应用代码



JUnit提供了简洁而清晰的测试结构，十分便于研发人员学习和使用

该框架使用注释来识别测试方法，使用断言来判断运行结果，同时还提供了测试运行机制来自动运行测试用例

在运行过程中，该框架可以实时反馈每一个测试用例的运行结果。该框架自动提供程序在每一个测试用例上的运行结果，无需人工检测生成测试报告



Junit示例

- 一个Junit实例

S0	package net.moocTest;
S1	public class Calculate {
S2	public int add(int a, int b) {
S3	return a+b;
S4	}
S5	public int subtract(int a, int b) {
S6	return a-b;
S7	}
S8	public int multiply(int a, int b) {
S9	return a*b;
S10	}
S11	public int divide(int a, int b) {
S12	return a/b;
S13	}
S14	}

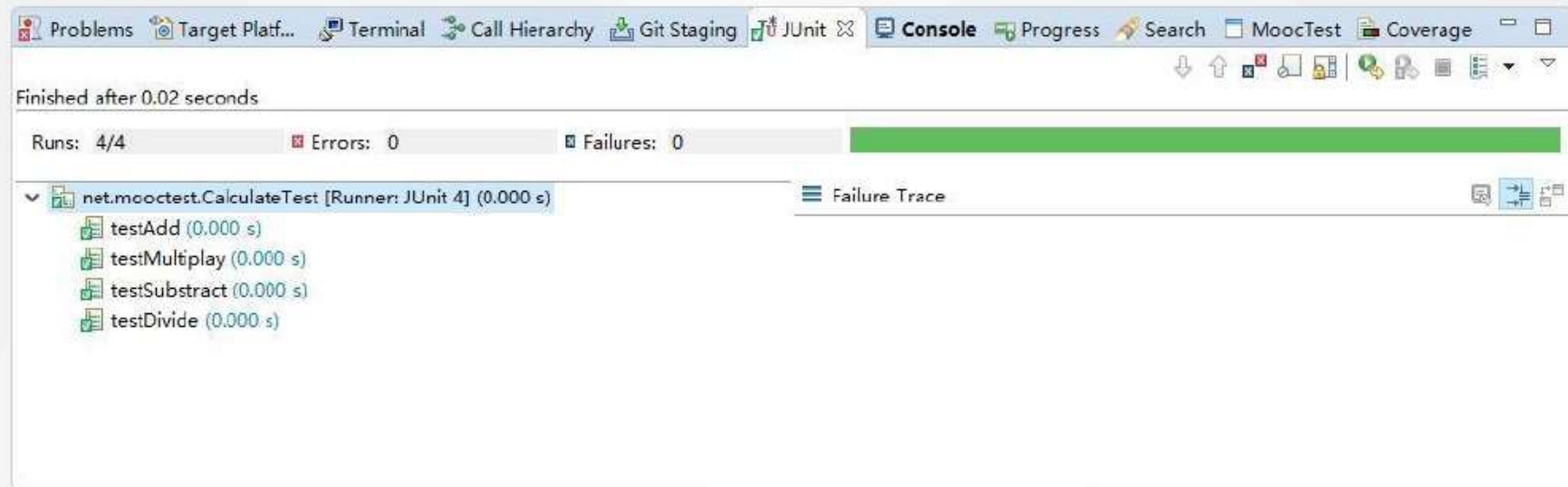
图 7-1 类Calculate

S0	package net.moocTest;
S1	import static org.junit.Assert.*;
S2	public class CalculateTest {
S3	@Test
S4	public void testAdd() {
S5	assertEquals(6,new Calculate().add(3, 3));
S6	}
S7	@Test
S8	public void testSubtract() {
S9	assertEquals(1,new Calculate().subtract(4, 3));
S10	}
S11	@Test
S12	public void testMultiplay() {
S13	assertEquals(12,new Calculate().multiplay(3, 4));
S14	}
S15	@Test
S16	public void testDivide() {
S17	assertEquals(2,new Calculate().divide(6, 3));
S18	}
S19	}

图 7-6 添加测试用例的测试类CalculateTest



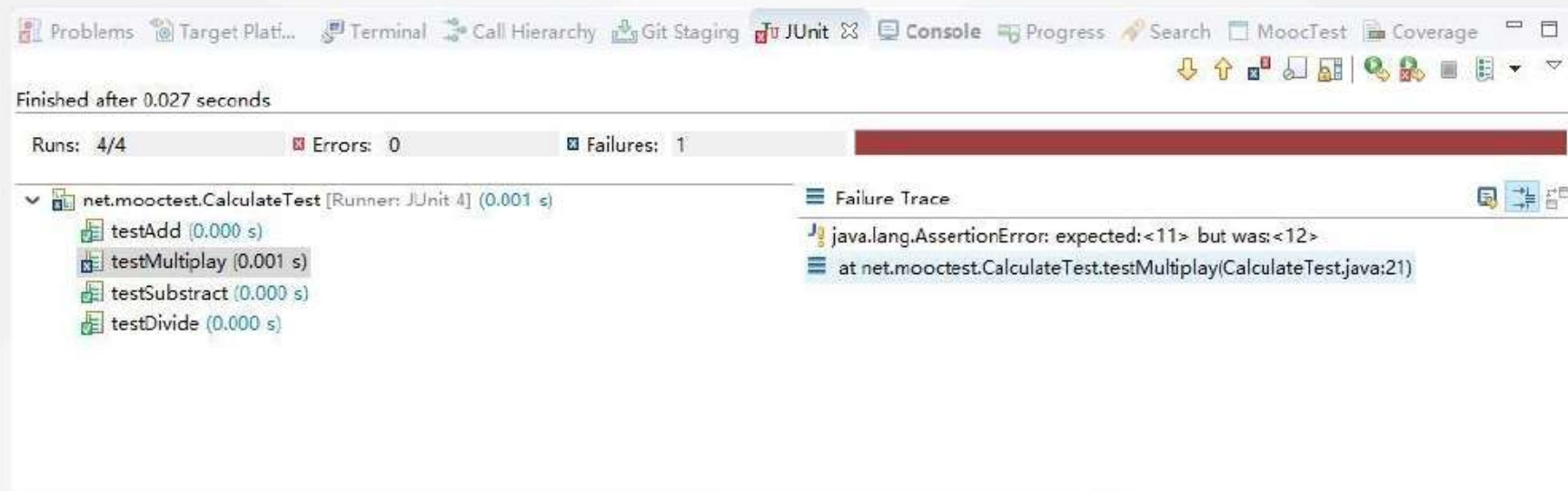
Junit示例



(1) 测试用例运行成功



Junit示例



(2) 测试用例运行失败



Junit示例

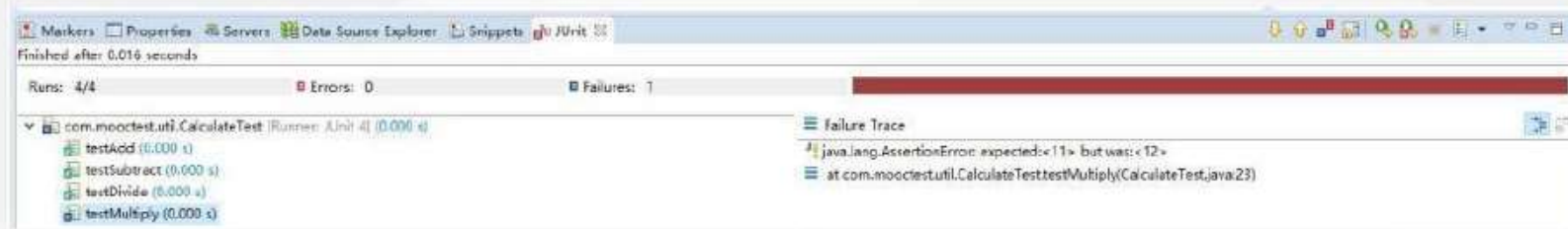
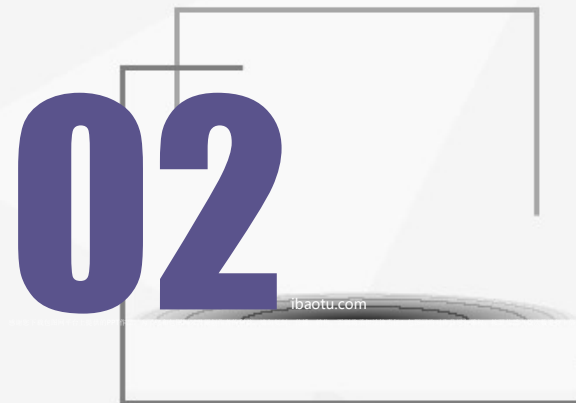


图 7-8 JUnit报错信息



注解

注解



注解是简化软件控制结构，提高程序自动化水平的重要方法。JUnit也提供了注解功能，帮助用户更清晰地表达测试程序的逻辑结构和功能



常用的JUnit注解包括@BeforeClass、@AfterClass、@Before、@After、@Test、@Ignore等，它们规定了每个测试用例的运行次序，即@BeforeClass→@Before→@Test→@After→@AfterClass，从而确定了整个测试流程



注解



@BeforeClass

@BeforeClass所注解的方法是JUnit测试时首个被运行的方法，且只被运行一次

baomidou.com



@AfterClass

@AfterClass所注解的方法是JUnit测试时最后一个被运行的方法，且只被运行一次



@Before

@Before所注解的方法在每个测试用例运行之前运行，通用用于初始化测试用例所需的资源

注解



@After

@After所注解的方法在每个测试用例运行之后运行，通用用于释放@Before所注解的方法打开的资源。当@Test或@Before所注解的方法发生异常时，@After所注解的方法仍会被运行



@Test

@Test所注解的方法被称为测试用例，包含测试代码。包含expected和timeout两个可选参数。expected表示运行后应该抛出的异常；timeout表示运行时间



@Ignore

@Ignore所注解的方法在测试过程中不会运行

注解

s0	package net.moocTest;
s1	public class JunitThread extends Thread {
s2	public int add(int a, int b){
s3	int result = 0;
s4	try{
s5	sleep(1000);
s6	result = a + b;
s7	} catch(InterruptedException e) {}
s8	return result;
s9	}
s10	public int division(int a, int b){
s11	return a / b;
s12	} }

图 7-9 类JunitThread



注解

S0	package net.moocetest;
S1	import static org.junit.Assert.*;
S2	public class JunitThreadTest {
S3	@BeforeClass
S4	public static void setUpBeforeClass() throws Exception {
S5	System.out.println("In BeforeClass");
S6	}
S7	@AfterClass
S8	public static void tearDownAfterClass() throws Exception {
S9	System.out.println("In AfterClass");
S10	}
S11	@Before
S12	public void before() { System.out.println("In Before"); }
S13	@After
S14	public void after() { System.out.println("In After"); }
S15	@Test(timeout=10000)
S16	public void testAdd() {
S17	System.out.println("In Test Add");
S18	assertEquals(6, new JDemo().add(3, 3));
S19	}
S20	@Test(expected=ArithmeticException.class)
S21	public void testDivision() {
S22	System.out.println("In Test Division");
S23	assertEquals(3, new JDemo().division(6, 2));
S24	assertEquals(3, new JDemo().division(6, 0));
S25	}
S26	@Ignore
S27	@Test
S28	public void testIgnore() {
S29	System.out.println("In Test Ignore");
S30	}
S31	}

图 7-10 测试类JunitThreadTest

In BeforeClass
In Before
In Test Add
In After
In Before
In Test Division
In After
In Before
In After
In AfterClass

图 7-11 运行结果



03

测试类与测试方法

目录

CONTENTS

PART 01 Junit示例

PART 02 注解

PART 03 测试类与测试方法

Assert

TestCase

TestResult

TestSuite

PART 03 错误与异常处理

PART 03 批量测试



Assert



检查并判断程序的运行结果是软件测试中的一项重要工作。在测试前测试者通常会在程序中埋入一些断言条件，若程序运行时不能满足这些条件，则表明程序的运行状态（或运行结果）与期望不一致，程序中存在缺陷



一般的，断言方法中的参数包括期望变量和实际变量两个部分。在断言语句运行时，若期望与实际的值相等，则表明程序运行结果与期望相符；否则，若期望与实际的值不相等，则表明程序运行结果与期望相异，测试用例运行失败



Assert

方法	描述
<code>assertNull(Object object)</code>	检查对象是否为空
<code>assertNotNull(Object object)</code>	检查对象是否不为空
<code>assertEquals(long expected, long actual)</code>	检查long类型的值是否相等
<code>assertEquals(double expected, double actual, double delta)</code>	检查指定精度的double值是否相等
<code>assertFalse(boolean condition)</code>	检查条件是否为假
<code>assertTrue(boolean condition)</code>	检查条件是否为真
<code>assertSame(Object expected, Object actual)</code>	检查两个变量是否引用同一对象
<code>assertNotSame(Object unexpected, Object actual)</code>	检查两个变量是否不引用同一对象

表 7-1 Assert类中的主要方法

从JUnit4版本开始JUnit还引入了更强大的断言方法**assertThat**。**assertThat**断言通过引入Hamcrest的**Matcher**匹配符来提供更加灵活断言操作

同时，不同于**assert**断言所采用的“谓宾主”语法模式，**assertThat**断言采用了“主谓宾”的语法模式，使测试代码语法更符合逻辑，有效提高了代码的可读性



Assert

一般匹配符



allOf表示所有条件都成立测试才能通过，相当于“与”逻辑。例如对于下列语句，当number大于8且小于16时测试用例运行成功

- `assertThat(number, allOf(greaterThan(8), lessThan(16)));`



anyOf表示任一条件成立测试就能通过，相当于“或”逻辑。例如对于下列语句，当number大于16或小于8时测试用例运行成功

- `assertThat(number, anyOf(greaterThan(16), lessThan(8)));`



anything表示对于任一条件测试都能通过。示例如下：

- `assertThat(number, anything());`



Assert

一般匹配符



is表示参数值相等时测试才能通过。例如对于下列语句，当str的值为“test”时测试用例运行成功

- `assertThat(str, is( "test" ));`



not表示参数值不相等时测试才能通过。例如对于下列语句，当str的值不是“test”时测试用例运行成功

- `assertThat(str, not( "test" ));`



Assert

字符串相关匹配符



containsString表示参数包含给定字符串时测试才能通过。例如对于下列语句，当str包含字符串“test”时测试用例运行成功

- `assertThat(str, containsString( "test" ));`



endsWith表示参数以给定字符串结尾时测试才能通过。例如对于下列语句，当str以字符串“test”结尾时测试用例运行成功

- `assertThat(str, endsWith( "test" ));`



startsWith表示参数以给定字符串开始时测试才能通过。例如对于下列语句，当str以字符串“test”开始时测试用例运行成功

- `assertThat(str, startsWith( "test" ));`



Assert

字符串相关匹配符



`equalTo`表示参数值相等时测试才能通过。需要注意的是，该匹配符适用于字符串、数值、对象间的比较，相当于Object类的`equals`函数

- `assertThat(str, equalTo( "test" ));`



`equalToIgnoringCase`表示在忽略大小写的情况下，参数值相等时测试才能通过

- `assertThat(str, equalToIgnoringCase( "test" ));`



`equalToIgnoringWhiteSpace`表示在忽略字符串头尾的空格后，参数值相等时测试才能通过。需要注意的是，字符串中间的空格不能被忽略

- `assertThat(str, equalToIgnoringWhiteSpace( "test" ));`



Assert

数值相关匹配符



closeTo表示浮点类型参数相差在一定范围内测试就能通过。例如对于下列语句, num的值在[19.5, 20.5]范围内测试用例就能通过

- `assertThat(num, closeTo(20.0, 0.5));`



greaterThan表示数值参数大于给定的数值测试就能通过。例如对于下列语句, num的值大于16.0测试用例就能通过

- `assertThat(num, greaterThan(16.0));`



lessThan表示数值参数小于给定的数值测试就能通过。例如对于下列语句, num的值小于8.0测试用例就能通过

- `assertThat(num, lessThan(8.0));`



Assert

数值相关匹配符



`greaterThanOrEqualTo`表示数值参数大于等于给定的数值测试就能通过。例如对于下列语句，`num`的值大于等于16.0测试用例就能通过

- `assertThat(num, greaterThanOrEqualTo(16.0));`



`lessThanOrEqualTo`表示数值参数小于等于给定的数值测试就能通过。例如对于下列语句，`num`的值小于等于8.0测试用例就能通过

- `assertThat(num, lessThanOrEqualTo(8.0));`



Assert

Collection相关匹配符



hasEntry表示给定Map对象包含一组特定的key值和value值时测试才能通过。例如对于下列语句，若map对象包含键值对`< "x" , "y" >`，测试用例运行通过

- `assertThat(map, hasEntry( "x" , "y" ));`



hasItem表示对象包含特定元素时测试就能通过。例如对于下列语句，若obj对象包含元素element，测试用例运行通过

- `assertThat(obj, hasItem(element));`



Assert

Collection相关匹配符



hasKey表示Map对象包含特定的key值时测试才能通过。例如对于下列语句，若map对象包含键值“x”，测试用例运行通过

- `assertThat(map, hasKey( "x" ));`



hasValue表示Map对象包含特定的value值时测试才能通过。例如对于下列语句，若map对象包含值“y”，测试用例运行通过

- `assertThat(map, hasValue( "y" ));`

目录

CONTENTS

PART 01 Junit示例

PART 02 注解

PART 03 测试类与测试方法

Assert

TestCase

TestResult

TestSuite

PART 03 错误与异常处理

PART 03 批量测试

TestCase



TestCase类是JUnit框架中的核心类，在单元测试时，测试者编写的测试类均需直接或间接继承于TestCase类，依靠其提供的方法来实现测试用例的运行与判断



需要注意的是，TestCase类与测试用例的含义是不同的

- 测试用例通常是指单个测试，用以验证程序某个功能或某个单元模块的正确性。在JUnit框架中，测试用例一般指的是@Test所注解的测试方法
- 测试类继承自TestCase，包含了多个测试用例。因此，可以理解为TestCase是为测试用例提供服务的类

目录

CONTENTS

PART 01 Junit示例

PART 02 注解

PART 03 测试类与测试方法

Assert

TestCase

TestResult

TestSuite

PART 03 错误与异常处理

PART 03 批量测试



TestResult



TestResult类收集并记录所有测试用例的运行结果



测试用例的运行结果可分为成功、失败、错误等三类：

- 成功表示程序在运行时满足断言语句中的各项需求
- 失败表示程序在运行时不能满足断言语句中的各项需求
- 错误则是指程序在运行时发生了不可预料的问题

目录

CONTENTS

PART 01 Junit示例

PART 02 注解

PART 03 测试类与测试方法

Assert

TestCase

TestResult

TestSuite

PART 03 错误与异常处理

PART 03 批量测试



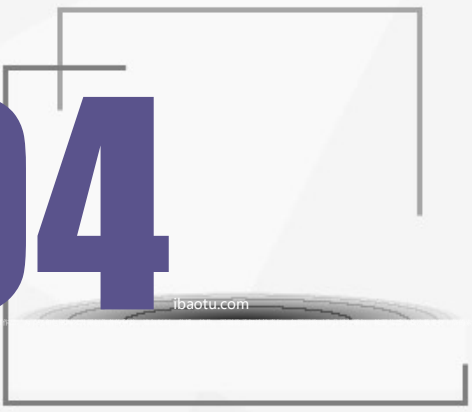
TestSuite



与TestCase类相同，TestSuite类也实现了JUnit中的Test接口，用于管理JUnit中的每一个测试用例



在单元测试过程中，即便没有显式构建TestSuite的子类，在运行TestCase子类时也会创建TestSuite类，并将每个TestCase子类的实例对象添加到TestSuite中运行



04

错误与异常处理

错误与异常

JUnit框架存在失败（Failure）和错误（Error）两种报错模式



失败由测试用例断言语句中的条件未能得到满足所引发，意味着程序运行状态或运行结果与预期不一致



错误则是由源程序或测试程序中的代码异常所引发，表示程序在测试用例运行时产生了意想不到的错误状态

异常处理

开发者在Java编程时会对程序中的异常进行处理



一般的，可以在发生异常的方法内添加try/catch/finally等处理语句，将异常在方法内部解决；也可以使用throw/throws等语句交由方法上一层进行解决



因此，在单元测试时也应当考虑方法抛出异常的情况，对异常的处理也应当纳入单元测试框架中



JUnit提供了多种异常的处理机制，帮助测试者验证需要异常处理的代码

异常处理

s0	public boolean canVote(int age){
s1	if(i<=0) throw new IllegalArgumentException("age should be +ve");
s2	if(i<18)
s3	return false;
s4	else
s5	return true;
s6	}

ibaozu.com

@Test(expected)

@Test注解中的参数expected允许测试者设置Throwable的子类，用于检测待测方法是否正确抛出异常

s0	@Test(expected=IllegalArgumentException.class)
s1	public void testCanVote1() {
s2	Student student = new Student();
s3	student.canVote(0);
s4	}

图 7-17 @Test(expected)示例

异常处理

ExpectedException

JUnit中的ExpectedException可以更精确地定位异常发生位置。在应用ExpectedException类前，需要用@Rule注解声明ExpectedException异常。然后，在异常发生位置前调用expect方法检测异常。此外，还可使用Expe

S0	@Rule
S1	public ExpectedException thrown = ExpectedException.none();
S2	@Test
S3	public void testCanVote2() {
S4	Student student = new Student();
S5	thrown.expect(IllegalArgumentException.class);
S6	thrown.expectMessage("age should be +ve");
S7	student.canVote(0);
S8	}

图 7-18 ExpectedException示例



异常处理

- **try/catch/fail**

S0	@Test
S1	public void testCanVote3() {
S2	Student student = new Student();
S3	try {
S4	student.canVote(0);
S5	} catch(IllegalArgumentException ex) {
S6	assertThat(ex.getMessage(), containsString("age should be + ve"));
S7	}
S8	fail("expected IllegalArgumentException for non +ve age");
S9	}

图 7-19 try/catch/fail示例

05



批量测试



参数化测试



参数化测试功能允许测试者一次性输入多个参数及对应预期输出，对待测方法进行更加全面的测试。参数化测试的主要步骤如下：

- 应用注解@RunWith(Parameterized.class)对测试类进行注释
- 创建一个由@Parameters注释的公共的静态方法，该方法返回一个对象集合(数组)作为测试数据集合
- 创建一个公共的构造函数，用于接收参数输入和预期输出
- 为每一个测试数据创建一个实例变量
- 基于实例变量构建测试用例



打包测试



参数化测试常用于对待测类中的某个方法进行批量化测试。然而，一次测试单个方法或单个类效率过低。如果需要一次性测试多个类，可以使用JUnit提供的打包测试功能。打包测试的主要步骤如下：

- 应用@RunWith(Suite.class)对测试类进行注释
- 应用@SuiteClasses(test_classes)对测试类进行注释，其中test_classes表示所有的测试类

谢谢欣赏